



INSTITUTO FEDERAL
DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA

Instituto Federal do Sudeste de Minas Gerais, *Campus Barbacena*

Bacharelado em Sistemas de Informação

Desenvolvimento de Páginas Web I

Professor Rafael Alencar
rafael.alencar@ifsudestemg.edu.br

Declaração `@import`

Permite importar uma folha de estilo a partir de outra folha de estilo ou do bloco `<style>` no HTML. Exemplos:

Importação no arquivo CSS:

```
@import url("estilo2.css");

*{
    margin: 0;
    padding: 0;
}
```

Importação no arquivo HTML:

```
<style type="text/css">
    @import url("estilo2.css");
</style>
```

Declaração `!important`

Essa declaração força o CSS a utilizar a propriedade e o valor descritos na linha onde é aplicada. Como as folhas de estilo aplicam seu estilo em cascata, baseado em hierarquias, a propriedade `!important` permite ao desenvolvedor forçar que determinada regra prevaleça.

Regras de precedência das folhas de estilo, com prioridade decrescente:

- CSS inline
- Seletor de ID
- Pseudo seletor
- Seletor de atributo
- Seletores de classe
- Seletor universal

Exemplo forçando ignorar a precedência:

```
<p class="c" id="i">Parágrafo</p>

#i{
    background-color: red;
}

.c{
    background-color: blue !important;
}
```

Exemplo forçando ignorar CSS inline:

```
<p style="color: blue">Parágrafo</p>

p{
    color: green !important;
}
```

Função calc()

Permite definir valores CSS por meio de expressões matemáticas, envolvendo inclusive unidades de medidas diferentes, como pixels e porcentagem. É possível utilizar os operadores de adição, subtração, divisão e produto.

Veja um exemplo que Posiciona um bloco na parte inferior direita da tela. Observe que foram utilizadas as unidades de medida vw e vh, que representam, respectivamente, 1/100 da largura e da altura da área visível da tela (viewport):

```
div{
  background-color: green;
  width: 100px;
  height: 100px;
  margin-left: calc(100vw - 100px);
  margin-top: calc(100vh - 100px);
}
```

Propriedade display

Permite definir como o elemento será renderizado. Principais valores: none, block, inline e inline-block.

HTML base:

```
<p>Elemento do tipo bloco</p>
<strong>Elemento do tipo inline</strong>
<span>Elemento do tipo inline</span>
```

CSS base:

```
p, strong, span{
  border: solid 1px blue;
}
```

Exemplos de uso da propriedade display:

```
/* Ocultando o parágrafo: */

p{
  display: none;
}

/* Tornando o parágrafo um elemento inline, passando a ter o tamanho
do seu conteúdo e não mais 100% na horizontal: */

p{
  display: inline;
}
```

```
/* Atenção: elementos do tipo inline não suportam a propriedade width. Para corrigir isso, deve-se utilizar o valor inline-block: */
```

```
p{  
  display: inline;  
  width: 50%; /* Não funciona */  
}
```

```
p{  
  display: inline-block;  
  width: 50%;  
}
```

```
/* Renderizando o elemento span como bloco: */
```

```
span{  
  display: block;  
}
```

Propriedade `position`

Especifica o método de posicionamento do elemento. Principais valores:

- `static`: padrão, posiciona o elemento de acordo com o fluxo normal da página;
- `relative`: qualquer mudança no posicionamento é calculada a partir de sua posição normal;
- `absolute`: posiciona o elemento em relação a seu ancestral mais próximo;
- `fixed`: o elemento permanece fixo em relação ao viewport, independente da rolagem da página.

Uma vez definida a posição do elemento, e caso ela não seja o valor `static`, o mesmo pode ser movido por meio das propriedades `top`, `bottom`, `left` e `right`. Exemplos:

Cabeçalho fixo (HTML):

```
<header>
  <h1>Header fixo</h1>
</header>

<p>
  Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nunc
  porta mi sed ex efficitur, lobortis convallis nunc vehicula...
</p>
```

Cabeçalho fixo (CSS):

```
header{
  background-color: #2196F3;
  color: #fff;
  font-family: sans-serif;
  font-size: 14px;
  padding: 10px;
  text-align: center;
  width: 100%;
  position: fixed;
}

p{
  padding: 60px 15px 15px;
  text-align: justify;
  line-height: 25px;
}
```

Botão fixo (HTML):

```
<button>+</button>
```

Botão fixo (CSS):

```
button{
  background-color: #F44336;
  color: #fff;
  font-size: 25px;
  width: 50px;
  height: 50px;
  border: none;
  border-radius: 50%;
}
```

```
    box-shadow: 1px 1px 5px #ddd;
    cursor: pointer;
    position: fixed;
    right: 15px;
    bottom: 15px;
}
```

Meta tag viewport

Com a variedade de tamanhos e resoluções de tela dos dispositivos móveis, o layout de sites planejados apenas para navegadores desktop acabam ficando “quebrados” nesses dispositivos. A tag meta `viewport` permite controlar como o navegador irá definir as dimensões e escala da tela.

Uso básico da meta tag, dentro do cabeçalho (head) da página:

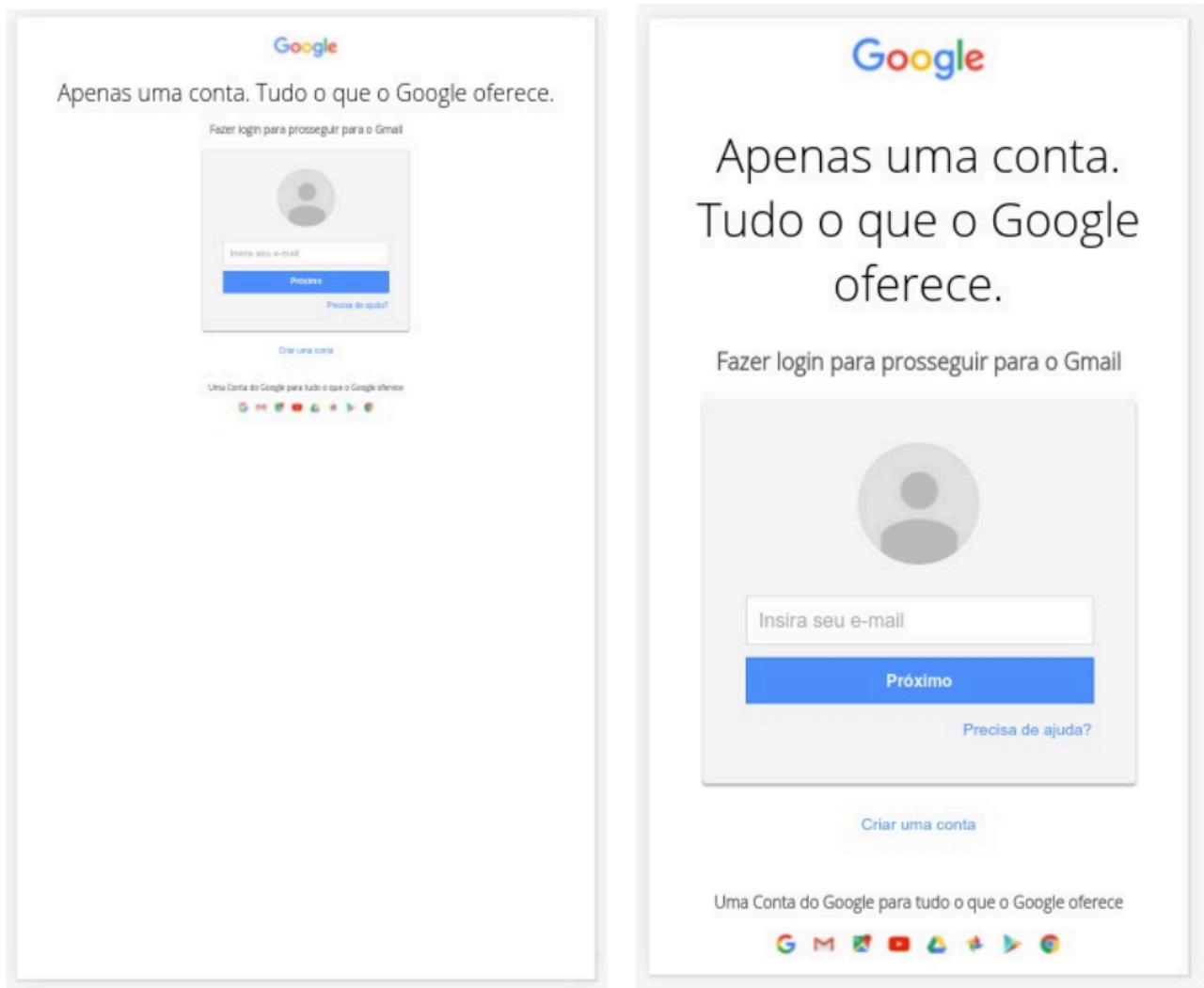
```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

O valor `device-width` indica ao navegador que a página deverá ser renderizada utilizando como largura (width) a largura da tela, seja de um celular, tablet ou desktop. Já o valor `1.0` do atributo `initial-scale` define o zoom inicial da página (no caso, sem zoom inicial).

É possível impedir que o usuário altere o zoom da página, adicionando a propriedade `user-scalable`:

```
<meta name="viewport"
      content="width=device-width, initial-scale=1.0, user-scalable=no">
```

Exemplo de uma página de login renderizada em dispositivo móvel sem e com a meta tag `viewport`, respectivamente:



Media queries

Permitem aplicar determinados estilos CSS dependendo das dimensões da tela do dispositivo (desktop, celular, tablet) e sua orientação (retrato ou paisagem). Dessa forma, é possível criar sites com layout adaptativo, visualmente agradáveis em diferentes dispositivos.

São utilizados **breakpoints** para determinar quando o layout da tela irá mudar. Exemplo de valores:

- Celulares: < 768px
- Tablets: > 768px
- Desktop: > 992px
- Desktop com telas maiores: > 1200px

Exemplo 1: CSS aplicado apenas em telas com tamanho máximo de 768px:

```
@media (max-width: 768px) {  
  body{  
    background-color: blue;  
  }  
}
```

Exemplo 2: CSS aplicado apenas em telas maiores que 992px:

```
@media (min-width: 992px) {  
  body{  
    background-color: green;  
  }  
}
```

Exemplo 3: CSS aplicado apenas em telas na orientação paisagem:

```
@media (orientation: landscape) {  
  body{  
    background-color: blue;  
  }  
}
```

Exemplo 4: CSS aplicado apenas em telas na orientação retrato:

```
@media (orientation: portrait) {  
  body{  
    background-color: blue;  
  }  
}
```

Exemplo 5: CSS aplicado apenas para impressão:

```
@media print {  
  aside{  
    display: none;  
  }  
}
```


No desenvolvimento do layout, podem ser utilizadas as abordagens **desktop first** (código para desktop fora das media queries) ou **mobile first** (código para mobile fora das media queries):

HTML do exemplo:

```
<div class="coluna"></div>
<div class="coluna"></div>
```

CSS desktop first:

```
.coluna{
  background-color: #ddd;
  margin: 15px 10px;
  border-radius: 10px;
  height: 100px;
  width: calc(50% - 20px);
  float: left;
}

@media (max-width: 768px){
  .coluna{
    float: none;
    width: calc(100% - 20px);
  }
}
```

CSS mobile first:

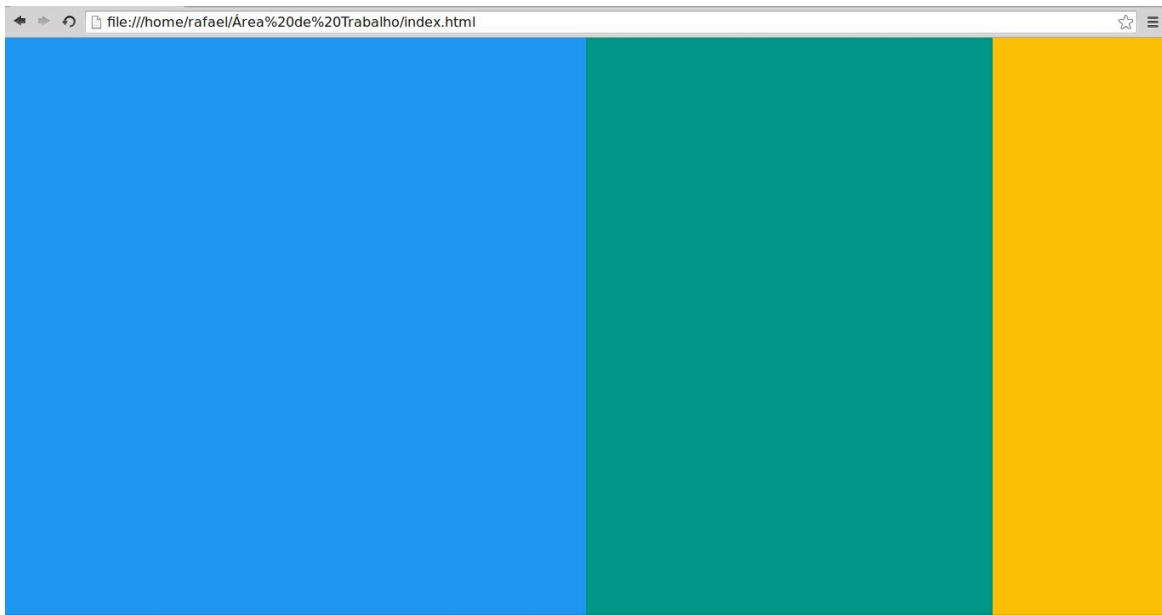
```
.coluna{
  background-color: #ddd;
  margin: 15px 10px;
  border-radius: 10px;
  height: 100px;
  width: calc(100% - 20px);
  float: none;
}

@media (min-width: 768px){
  .coluna{
    float: left;
    width: calc(50% - 20px);
  }
}
```

Atividade de fixação

Utilizando a abordagem **desktop first**, crie uma página com três colunas nas cores (#2196F3, #009688 e #FFC107), onde:

Em uma **tela desktop padrão (sem media queries)**, as colunas deverão ter como valor de largura, respectivamente, 50%, 35% e 15%, e 100vh de altura:



Caso a **tela possua até 1000px de largura**, a primeira coluna deverá possuir 60% da largura, a segunda coluna 40%, e a terceira coluna deverá descer, ficando com 100% de largura. As colunas deverão passar a ter 50vh de altura cada:



Caso a **largura da tela seja de até 500px**, as três colunas deverão ter 100% de largura, e, como altura, respectivamente, 50vh, 35vh e 15vh:

